SCHEDULED SCRIPT

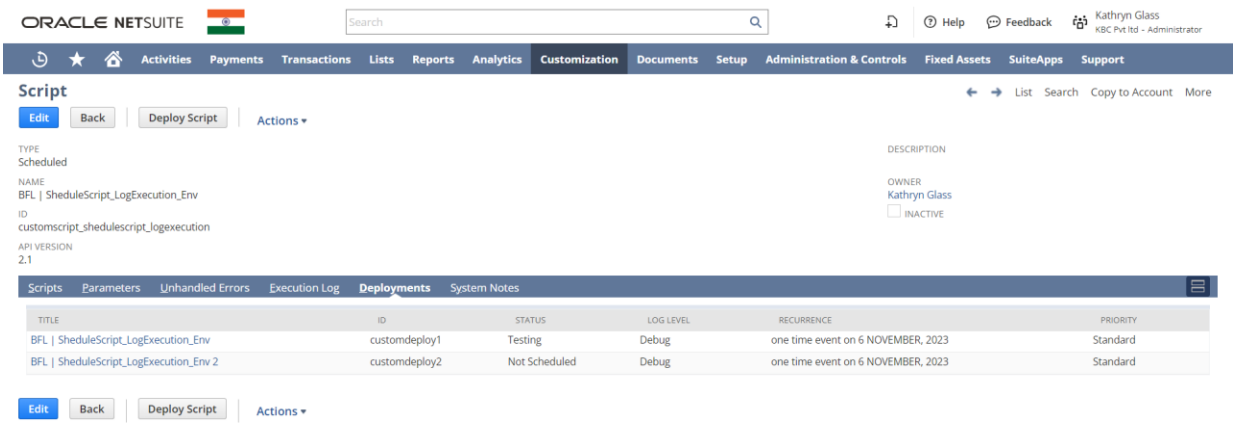
Q.1. Use log.debug in the scheduled script.

Ans.

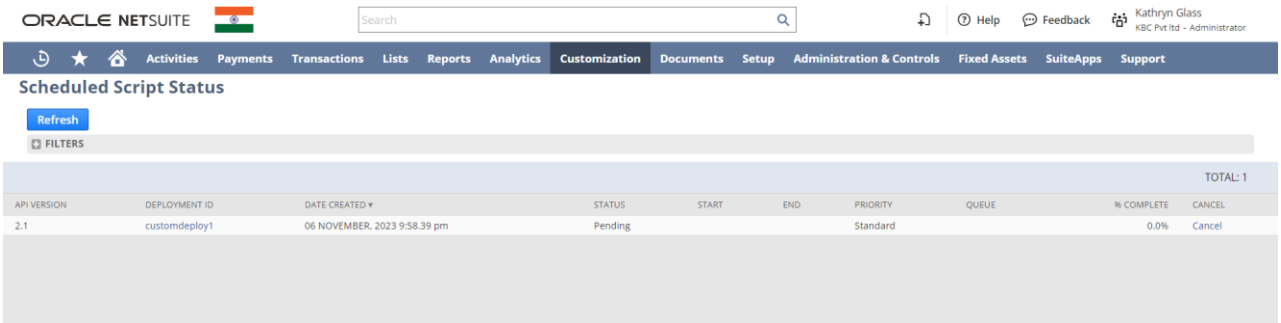
1.We will create a shedule script and use log.debug() to print log

```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 */
define(['N/currentRecord'],

    (currentRecord) => {

        const execute = (scriptContext) => {
            var now = new Date();
            var execution_Time = now.toISOString();
            log.debug({
                title : 'Shedule Script has been execute',
                details : 'Shedule script execute at' + execution_Time,
            });

        };

        return {execute};

});
```

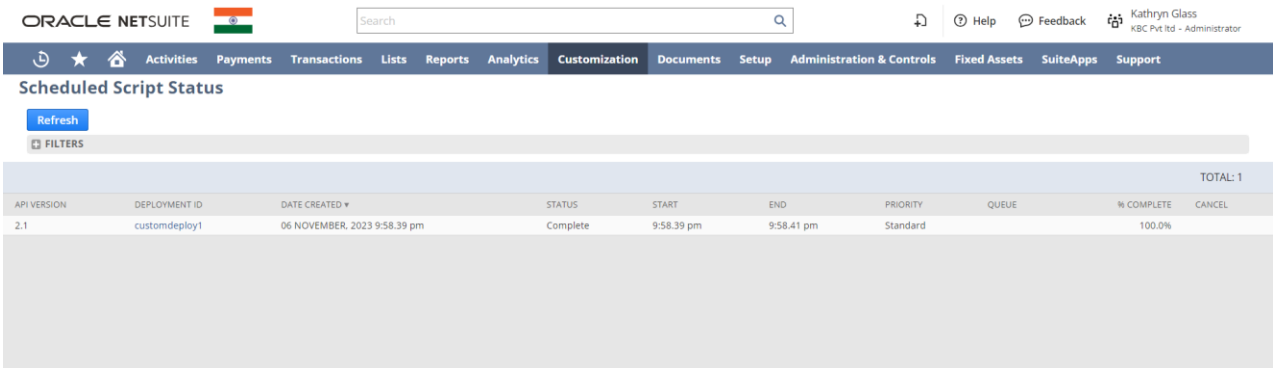
2.Script has been deployed



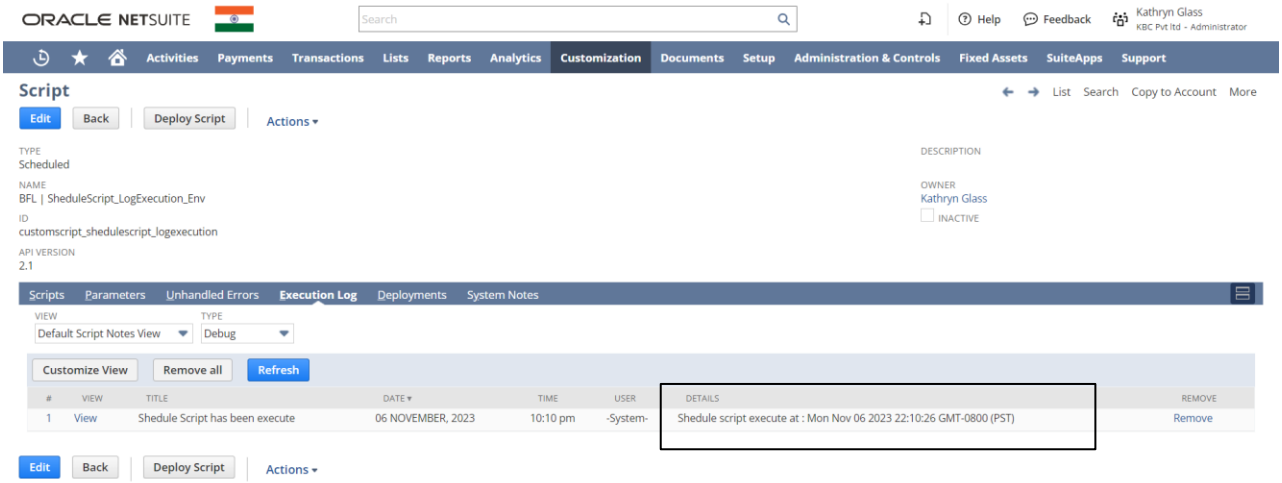
3.Script deployment is pending



4.Script deployment is complete



5.Log has been print in Execution log



Q.2. Created a parameter and retrieved the value from it.

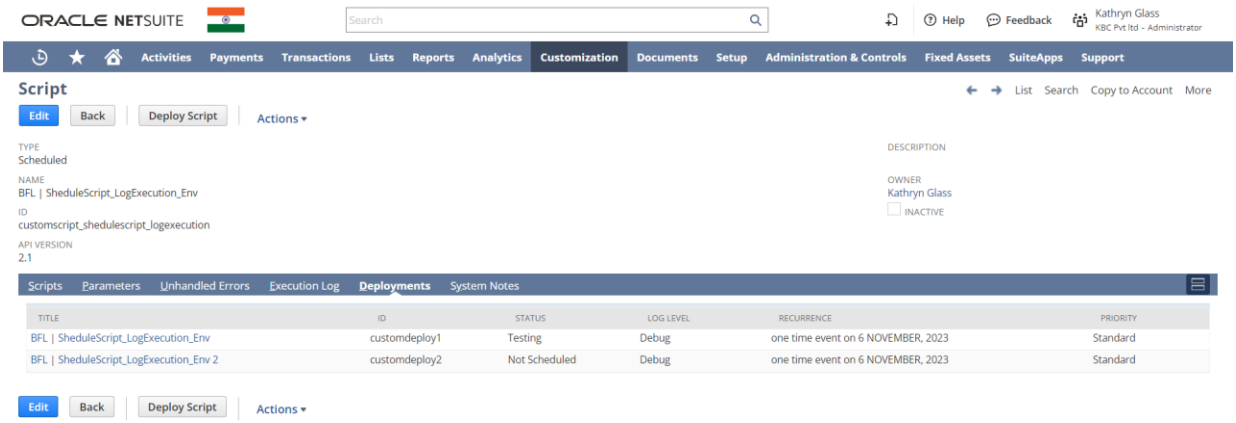
Ans.

1.We will create a shedule script and use log.debug() to print log

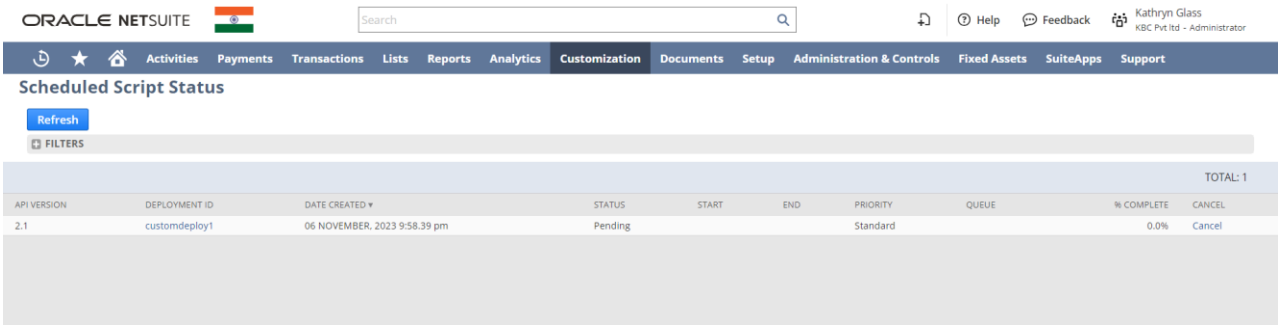
2.We use .getParameter() to fetch parameter using ‘name’ parameter

```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 */
define(['N/currentRecord' , 'N/runtime' , 'N/task'],

    (currentRecord, runtime) => {

        const execute = (scriptContext) => {
            // var now = new Date();
            // var execution_Time = now.toISOString();
            // log.debug({
                // title : 'Shedule Script has been execute',
                // details : 'Shedule script execute at' + execution_Time,
            // });

            var parameter_1 = runtime.getCurrentScript().getParameter({
                name : 'custscript_parameter_1',
            });
            log.debug({
                title : 'Shedule Script has been execute',
                details : 'Default Value of Parameter 1 is : ' + parameter_1,
            });



        };

        return {execute};

});
```

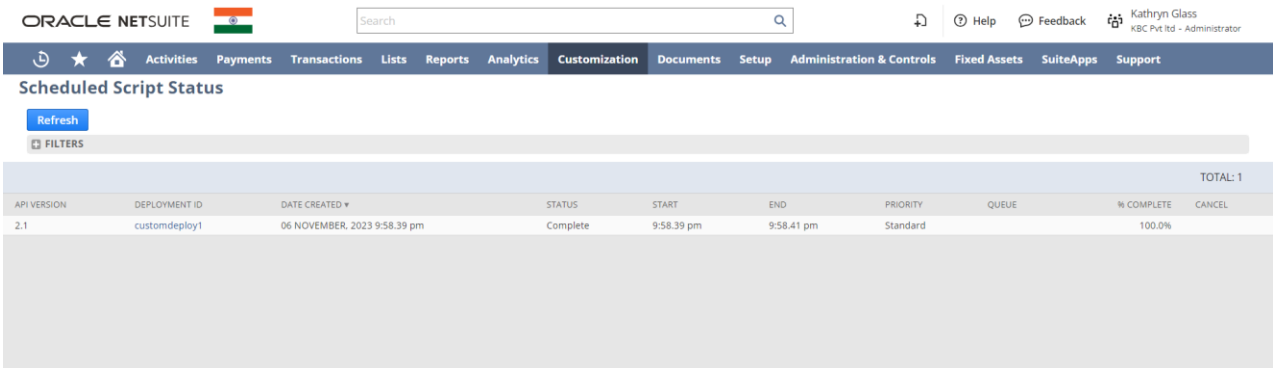
2.Script has been deployed



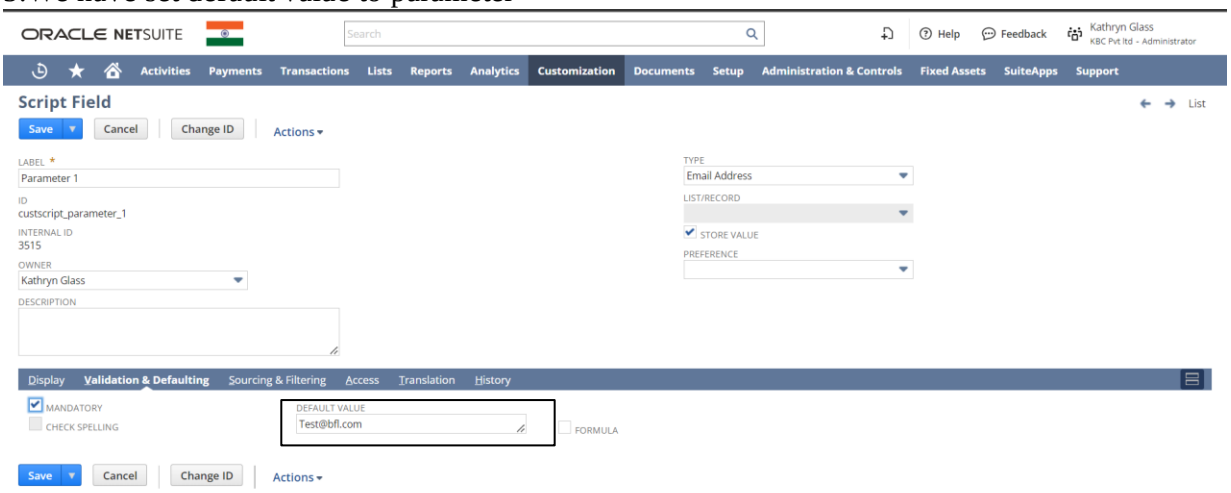
3.Script deployment is pending



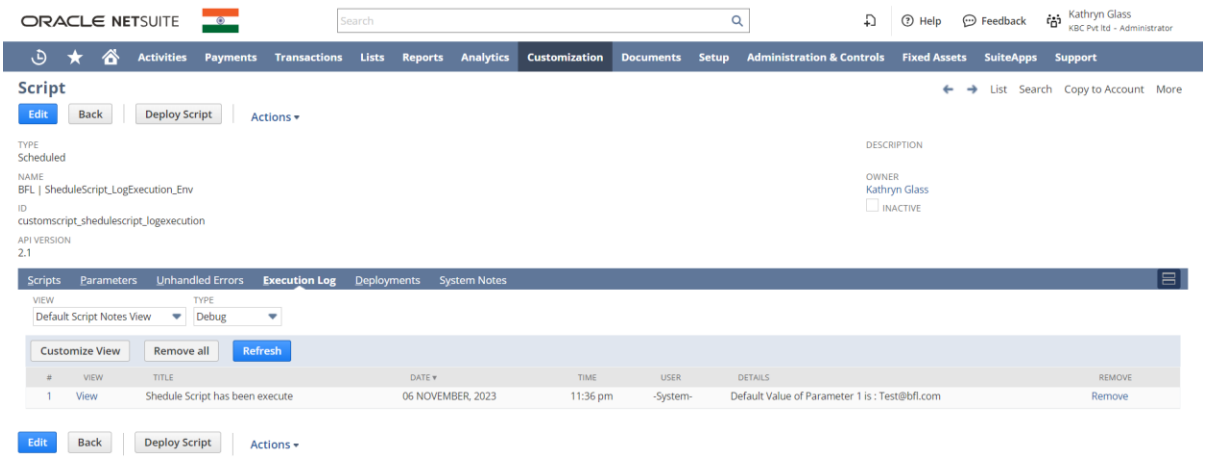
4.Script deployment is complete



5.We have set default value to parameter



6.set value has been print



Q.3. On the button click on the suitelet, scheduled script should get trigger.

Ans.

1.We have create Suitelet form to select customer

2.Here we will create customer name drop down and button to send email

3.Then we will create a task using task.create()

4.And call the schedule script

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/record', 'N/ui/serverWidget','N/http', 'N/task'],
    /**
 * @param{record} record
 * @param{serverWidget} serverWidget
 */
    (record, serverWidget, http, task) => {

       function onRequest(scriptContext) {
             if (scriptContext.request.method === 'GET'){
                let form = serverWidget.createForm({
                title: 'Customer Form'
            });

            let select = form.addField({
                id: 'custpage_customer_name',
                type: serverWidget.FieldType.SELECT,
                label: 'Customer',
                source: 'customer'
            });

            form.addSubmitButton({
                label: 'Send Mail'
            });

            scriptContext.response.writePage(form);
            }
            else if (scriptContext.request.method === 'POST')
            {
                    const delimiter = /\u0001/;
```

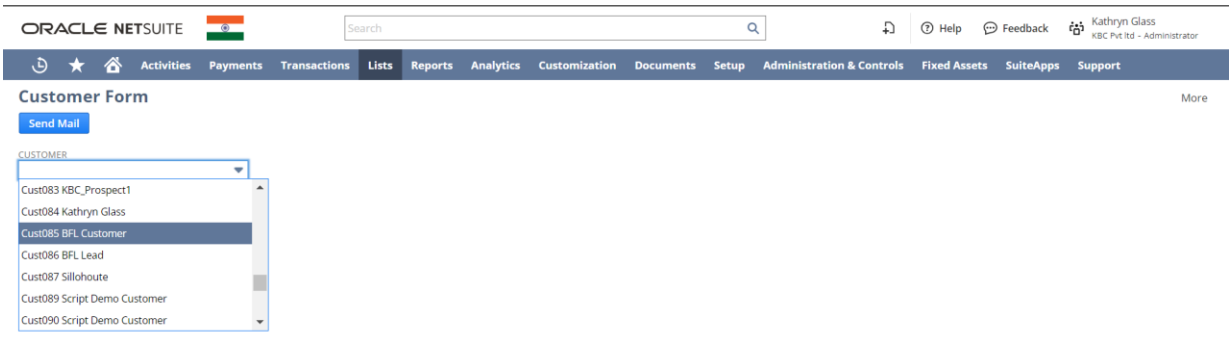
```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 */
define(['N/currentRecord' , 'N/runtime' , 'N/task' , 'N/email' ],

    (currentRecord, runtime , email) => {

        const execute = (scriptContext) => {

            var customer_id = parseInt(runtime.getCurrentScript().getParameter({
                name : 'custscript_customer_name',
            }))
            log.debug({
                title : 'Shedule Script has been execute',
                details : customer_id,
            });
            email.send({
                author: -5,
                recipients: customer_id,
                subject: 'This Email has been send by Kathryn Glass through shedule mail',
                body: 'Suitelet-Shedule Script',


            });



        };

        return {execute};

});
```

4.Schedule script to store customer id and send email

5.here we will use .getParameter() to fetch parameter and customer will be save in this parameter

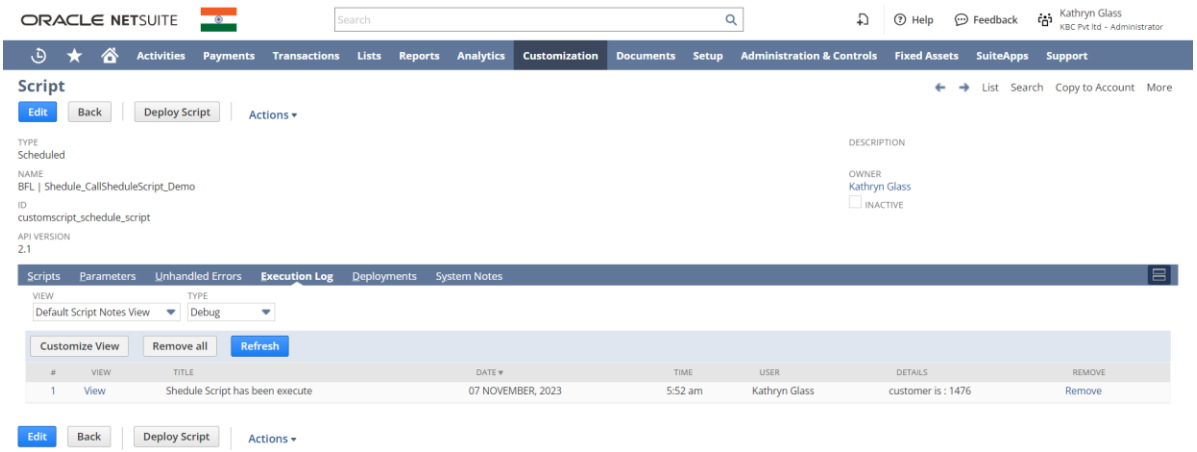
```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 */
define(['N/currentRecord' , 'N/runtime' , 'N/task' , 'N/email' ],

    (currentRecord, runtime , email) => {

        const execute = (scriptContext) => {

            var customer_id = parseInt(runtime.getCurrentScript().getParameter({
                name : 'custscript_customer_name',
            }))
            log.debug({
                title : 'Shedule Script has been execute',
                details : customer_id,
            });
            email.send({
                author: -5,
                recipients: customer_id,
                subject: 'This Email has been send by Kathryn Glass through shedule mail',
                body: 'Suitelet-Shedule Script',


            });



        };

        return {execute};

});
```

5.SuiteLet form



```
customer name 1476
```

5.Customer id has been saved in parameter of shedule script



Q.4. Create a scheduled script and make four deployments of the script. Print a different message on memo field through each deployment.

Ans.

1.Schedule script has been created to set memo on sales order

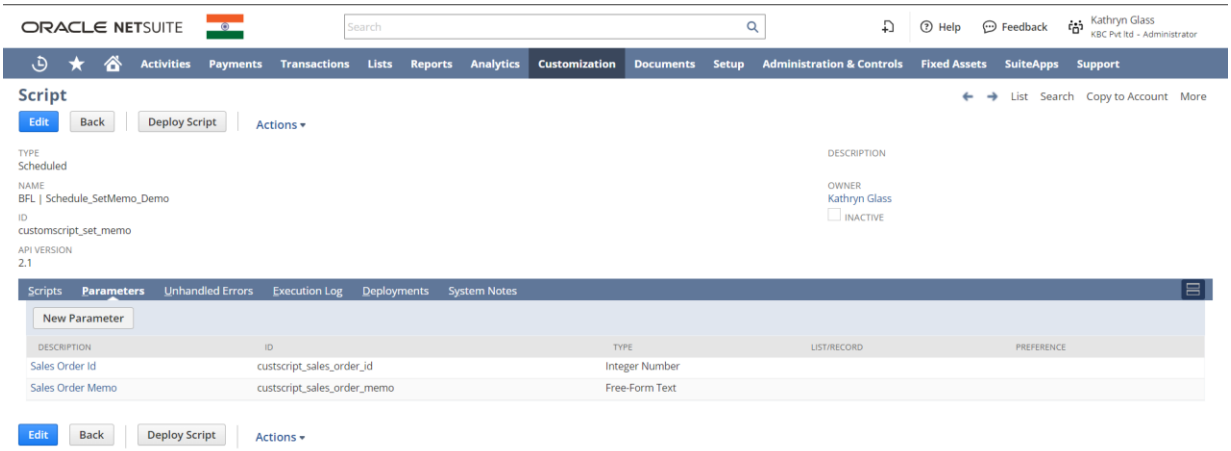
2.We will use .getParameter() to fetch parameters i.e. sales order id and sales order memo

3.Then we will load sales order using record.load()

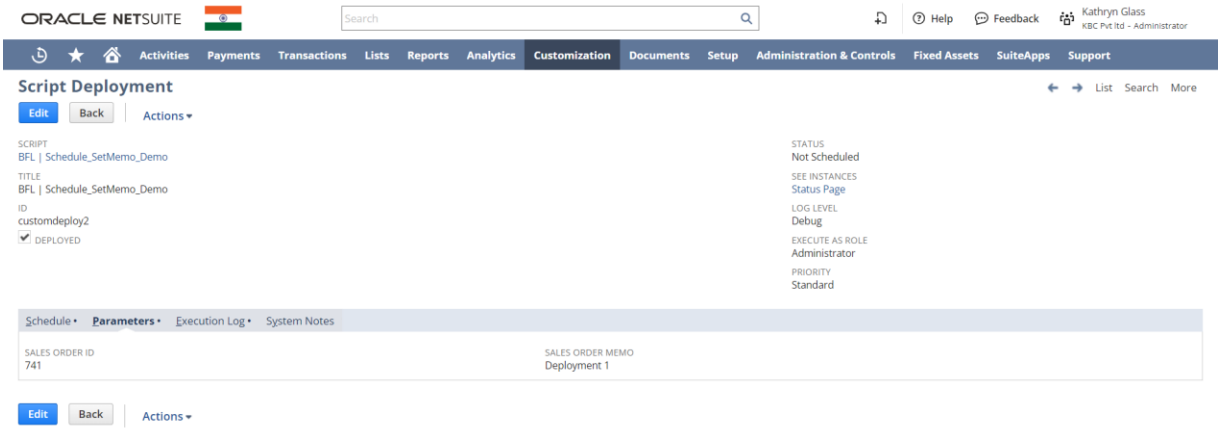
4.We will use salesorder.setValue() to set value in memo

```
/**
* @NApiVersion 2.1
* @NScriptType ScheduledScript
*/
define(["N/log", "N/record", "N/runtime", "N/search"],
(log, record, runtime, search) => {

  const execute = (scriptContext) => {
      var salesorder_id = runtime.getCurrentScript().getParameter({
          name: 'custscript_sales_order_id'
      });
      var salesorder_memo = runtime.getCurrentScript().getParameter({
          name: 'custscript_sales_order_memo'
      });
      var salesorder = record.load({
          type: record.Type.SALES_ORDER,
          id: salesorder_id,
          isDynamic: true,
      })
      log.debug('Sales Order Loaded Success : ', salesorder_id);
      salesorder.setValue({
          fieldId: 'memo',
          value: salesorder_memo,
      })
      salesorder.save({
          enableSourcing: true,
          ignoreMandatoryFields: true
      });
      log.debug('Memo has been saved: ' , salesorder_memo);
  };

  return { execute };
});
```

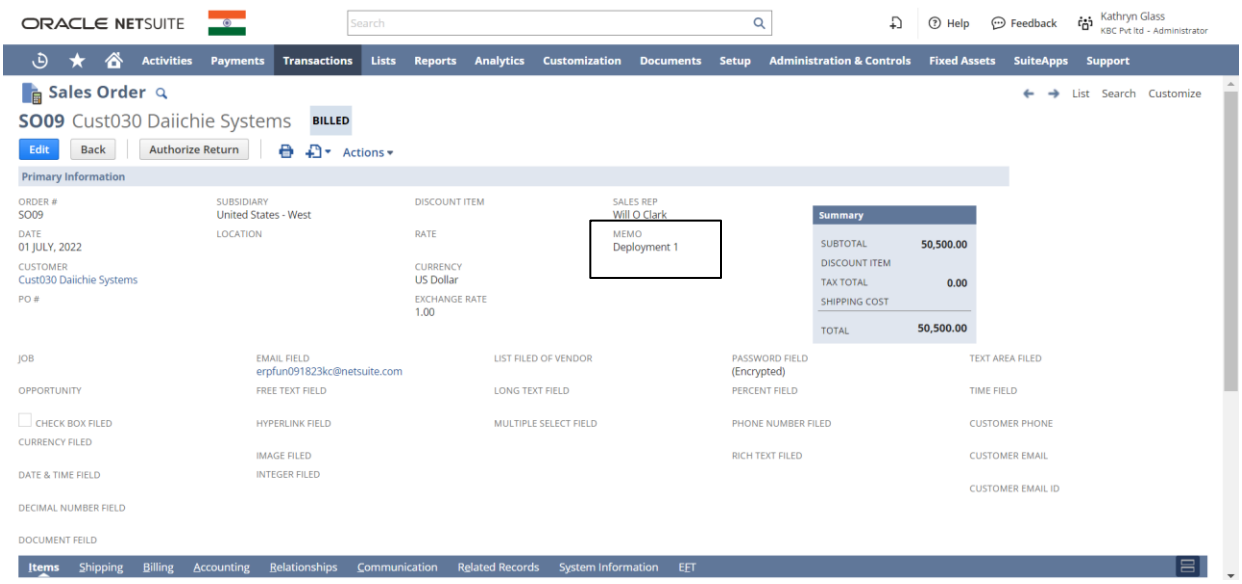
5.Script has been loaded and deploy



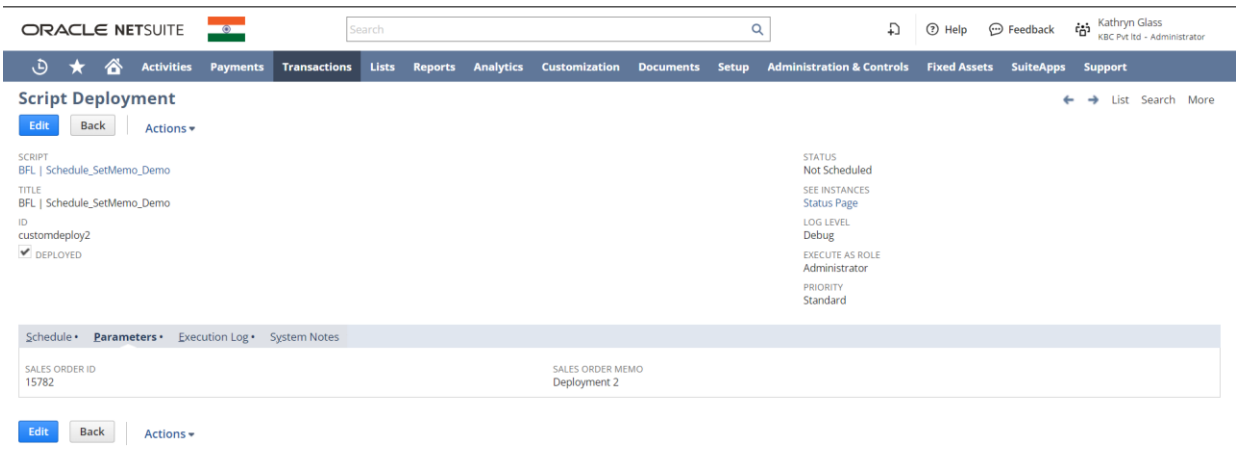
6.Deployment 1 to set memo on sales order using parameters



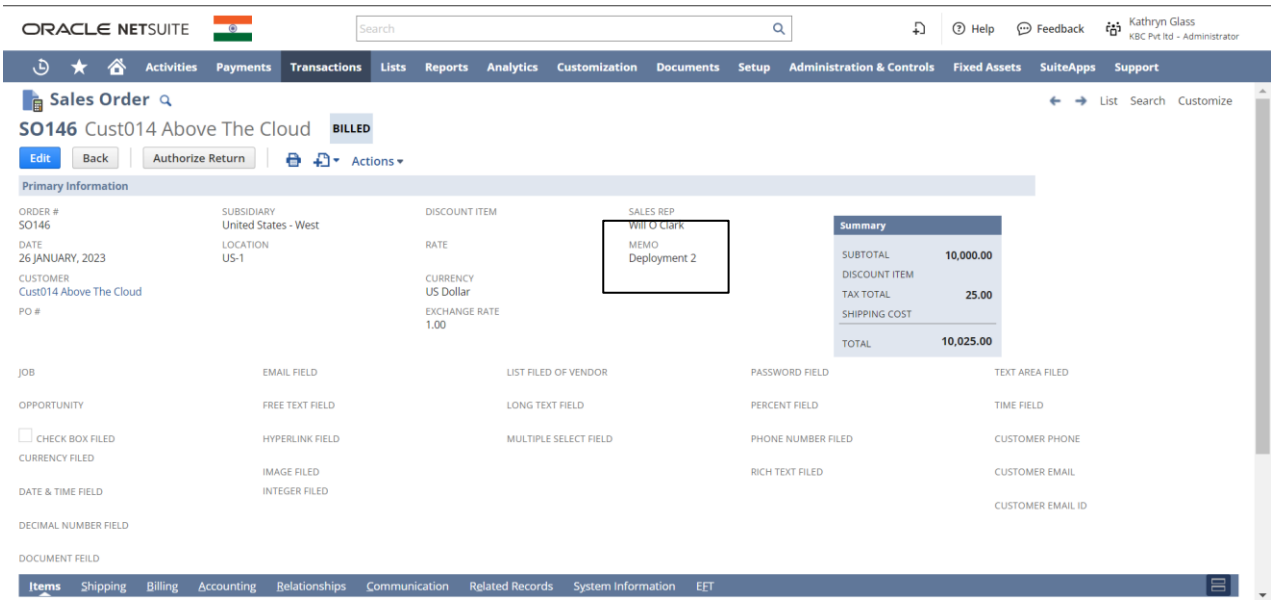
7.Memo has set on sales order ‘SO09’



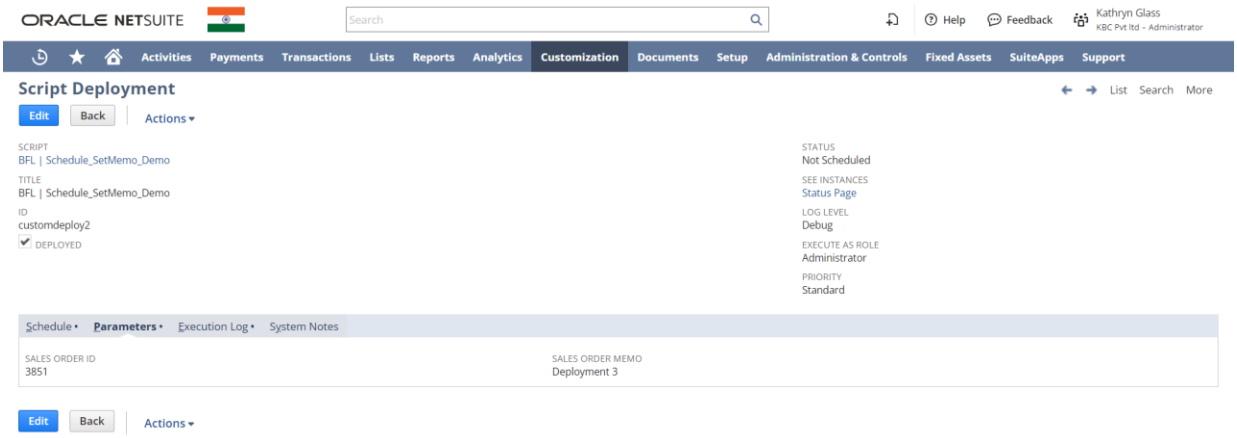
8.Deployment 2 to set memo on sales order using parameters



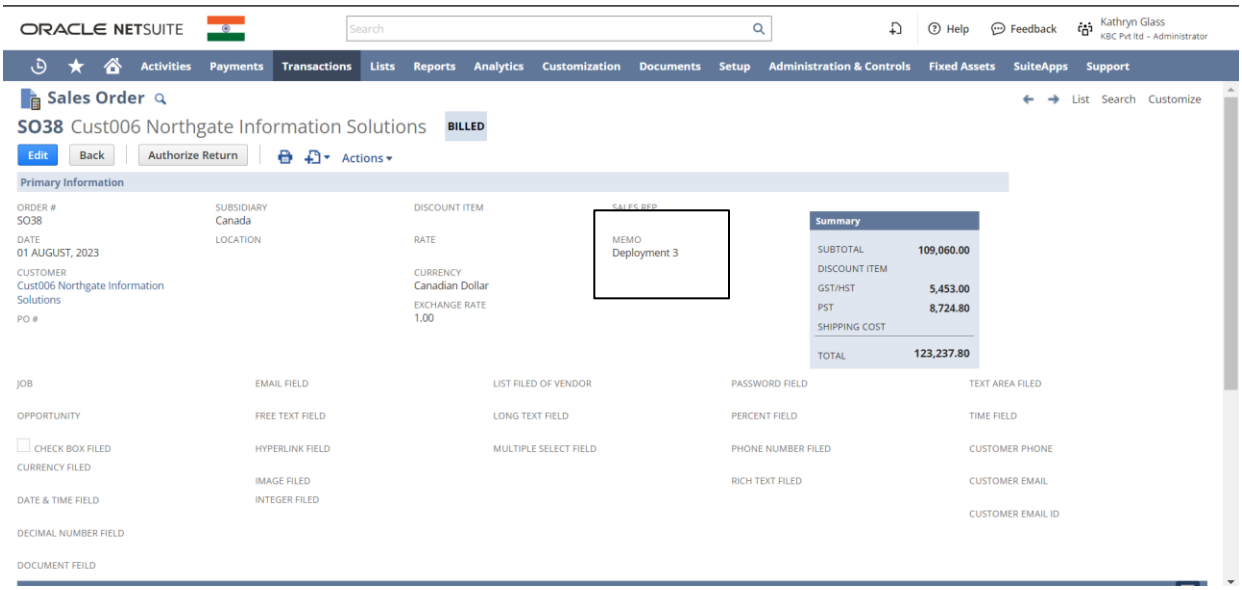
9.Memo has set on sales order ‘SO0146’



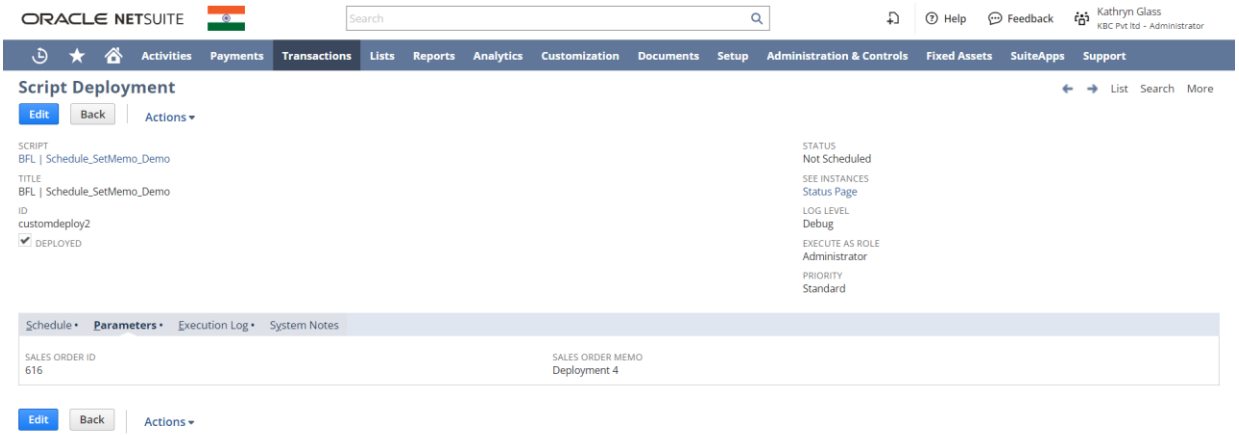
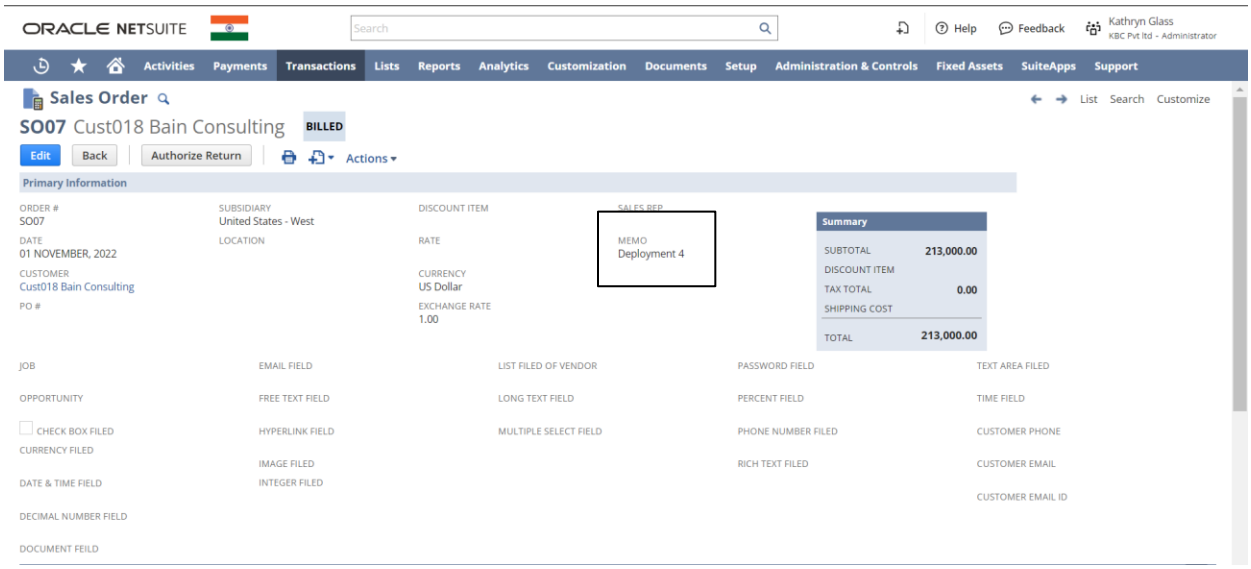
10.Deployment 3 to set memo on sales order using parameters



11.Memo has set on sales order ‘SO38’



12.Deployment 4 to set memo on sales order using parameters



13.Memo has set on sales order ‘SO07’